



# Clases en el desarrollo real

Propiedades, métodos, constructores y encapsulamiento — como se usan de verdad al diseñar software

---

*Sin perros que ladran. Con las mismas clases que vas a escribir en tu primer trabajo.*

# El vocabulario ya lo tienen



## Clase

Plantilla que define qué datos y qué comportamiento tendrá cada instancia.



## Objeto

Una instancia concreta de esa clase, con sus propios valores en memoria.



## Propiedad

Variable declarada dentro de la clase; guarda el estado del objeto.



## Método

Función declarada dentro de la clase; define una acción o un cálculo.



## Constructor

Método especial que se ejecuta al crear el objeto; inicializa su estado.



## Encapsulamiento

Acceso controlado a las propiedades (getter/setter), validando antes de modificar.

**Lo que falta:** saber CUÁNDO usar cada uno. Ahí es donde entra el diseño real de software.

# Los mismos 4 conceptos, en 3 lenguajes

## Java

```
class Producto {  
    private double precio;  
  
    public Producto(double p) {  
        precio = p;  
    }  
  
    public double getPrecio() {  
        return precio;  
    }  
}
```

## Python

```
class Producto:  
    def __init__(self, p):  
        self.precio = p  
  
    def get_precio(self):  
        return self.precio
```

## JavaScript

```
class Producto {  
    constructor(p) {  
        this.precio = p;  
    }  
  
    getPrecio() {  
        return this.precio;  
    }  
}
```

**La sintaxis cambia.** La propiedad precio, el constructor y el método siguen ahí en los 3 lenguajes — solo con llaves, indentación o palabras distintas.

# ¿Una clase siempre es una tabla?



class Usuario

```
class Usuario {  
  id  
  nombre  
  correo  
}
```



tabla "usuarios"

| id | nombre | correo  |
|----|--------|---------|
| 1  | Ana    | ana@... |



**Esta idea es correcta... solo para 1 de cada 3 tipos de clases que existen.** Si diseñan pensando que TODA clase debe reflejar una tabla, se les va a complicar el resto del sistema.

## 3 familias de clases en un sistema real



# Modelos (Datos / Entidades)

## ¿Qué son?

Clases que representan "cosas" del negocio. Tienen una relación directa (casi siempre 1 a 1) con una tabla de la base de datos.

## Su propósito

Almacenar información en memoria durante la ejecución del programa, lista para ser leída, validada o guardada.

## Ejemplos

Usuario

Factura

Producto

DetalleFactura



### Aquí dominan:



Propiedades: muchas. Son la razón de ser de la clase (id, nombre, precio...).



Constructor: inicializa esas propiedades al crear el objeto.



Getters/setters: validan antes de aceptar un valor (ej. precio  $\geq 0$ ).

# Servicios (Lógica de negocio)

## ¿Qué son?

Clases que NO tienen tabla en la base de datos. No guardan datos en propiedades; contienen métodos que ejecutan acciones.

## Su propósito

Procesar los objetos de la familia Modelos: aplicar reglas, hacer cálculos, o comunicarse con sistemas externos.

## Ejemplos

ProcesadorDePagos

GeneradorPdfFactura

ValidadorDeInventario



### Aquí dominan:



Métodos: son el protagonista. Casi siempre son verbos: procesar(), generar(), validar().



Propiedades: casi ninguna (a lo mucho, configuración fija).



Reciben objetos Modelo como parámetros; no los guardan, los transforman.

# Puentes (Controladores / Infraestructura)

## ¿Qué son?

La interfaz entre el mundo exterior (navegador, app móvil, API) y la aplicación. Tampoco se guardan en la base de datos.

## Su propósito

Recibir la petición, entender qué se pide, mandar a llamar a los Servicios y Modelos necesarios, y devolver una respuesta.

## Ejemplo

**FacturaController**



### Aquí dominan:



Métodos de orquestación: uno por cada acción que el usuario puede pedir.



Reciben, delegan a Servicios/Modelos, y responden. No hacen el trabajo pesado ellos mismos.



Propiedades: prácticamente ninguna propia del negocio.



## TODO JUNTO

# Un clic, varias clases trabajando en equipo

*Escenario: el cliente hace clic en "Emitir Factura Electrónica".*

1



**FacturaController**

PUENTE

Recibe el clic del usuario en el navegador con los datos del formulario.

2



**Factura / DetalleFactura**

MODELO

Se crean en memoria con los datos recibidos y se guardan en la base de datos.

3



**FacturacionHaciendaService**

SERVICIO

Convierte la factura al formato oficial, la firma digitalmente y la envía a Hacienda.

4



**NotificadorEmail**

SERVICIO

Si Hacienda aprueba, toma el PDF generado y envía la notificación al correo del cliente.

PARA QUE NO SE LES OLVIDE

# Propiedades, constructor y métodos, según la familia



## Modelos

### Propiedades

Muchas — son su razón de ser

### Constructor

Inicializa esas propiedades

### Métodos

Getters/setters y validaciones

### ¿Va a la BD?

Sí, casi siempre



## Servicios

### Propiedades

Casi ninguna

### Constructor

Rara vez (poco que inicializar)

### Métodos

Procesan, calculan, se comunican

### ¿Va a la BD?

No



## Puentes

### Propiedades

Prácticamente ninguna

### Constructor

Rara vez

### Métodos

Reciben, delegan y responden

### ¿Va a la BD?

No

## SÍNTESIS

# 80% estructura, 20% le da vida

80% — MODELOS

20% — SERVICIOS  
+ PUENTES



### Modelos: estructuran y guardan

La mayoría del tiempo de desarrollo se va en definir bien qué información existe en el sistema y cómo se relaciona entre sí.



### Servicios + Puentes: dan vida

Son menos clases, pero son las que hacen que la aplicación realmente procese, se comuniquen y responda al usuario.

## PARA RECORDAR

# Antes de crear una clase nueva, pregúntense:



¿Esta clase guarda datos que deben persistir?

→ *Es un Modelo.*



¿Esta clase ejecuta una acción o una regla de negocio?

→ *Es un Servicio.*



¿Esta clase recibe una petición externa y coordina a las demás?

→ *Es un Puente / Controlador.*



No todas las clases van a la base de datos. Y está perfecto que así sea — así es como se organiza el software de verdad.